MediBytes multilingual STT: proposed changes

Oct 14, 2026 · M Santhosh

The shipped pipeline is English-only end to end, and the repo holds no measured accuracy for Hindi, Tamil or code-mixed clinical speech. We propose seven changes in priority order, starting with measurement; every engine named here is a candidate to be decided on our own audio, not a verdict.

Where the code stands today

Nothing in the shipped chain is built for Hindi or Tamil; the only multilingual behaviour is a 13-word lookup table. The repo's own production plan describes a multilingual design that was never implemented.

Stage	What the code does today	Why it fails for hi-IN, ta-IN, Tanglish	Where
Language input	The HTTP job form takes `denoiser`, `model` and `use_llm`; there is no language field	The pipeline is never told the speaker's language	`server.py` create_job
Primary engine	`google/medasr`, an English-only dictation model with no language ID	The README states it "destroys code-mixed audio": `BP 130 by 80` became "BP aches or teaeth by a se"	`asr.py:109`, `README.md:189`
Fallback and second pass	Whisper `small-int8`, greedy by default, language auto-detected per clip; the second pass adds drug and vital rows from that decode	Small Whisper is weak on Indic speech, and clip-level detection is unreliable on mixed sentences; users must pass `--model small-int8` by hand for hi/ta	`decode_presets.py`, `stt_extract.py:218`, `README.md:190`
Extraction	`HINGLISH_MAP` has 13 entries (bukhar, khansi, din tak, a few Devanagari units); Tamil has none, only script detection	A perfect Tamil transcript in native script would yield no symptoms	`extract.py:13-18`, `extract.py:286`
Evaluation	The 90.2% fact accuracy is measured on English clips 008 to 030, the ones not used to develop the extractor; the four files of true transcripts and expected facts contain no Indic-script characters and no romanized-Hindi words	No hi/ta/Tanglish accuracy number exists anywhere in the repo	`docs/PIPELINE_EVAL_REPORT.md`, `clinical_eval/*.json`
Only hi/ta measurement	The denoising pilot's corpus had FLEURS Hindi and Tamil plus 3 Hindi-English clips; its no-denoise baseline scored WER 1.20	The report calls this recogniser failure, not a denoiser effect; the corpus is non-clinical	`docs/DENOISING_PILOT_20260927.md:18`
Planned design	The L7 plan specifies per-segment language ID, Whisper large-v3 as the base, Deepgram for Hindi, GCP Chirp 3 for Tamil, IndicXlit transliteration and MuRIL NER	None of these engines or models exists in `base_pipeline/`; the plan's Tamil claims also predate Deepgram's Tamil model	`PRODUCTION_PLAN_L7.md` §7

Proposed changes versus the current code

Seven changes, ranked. Change 1 blocks all the others, because without it no engine comparison can be trusted. Effort is my estimate, not a measured figure.

#	Change from today	Files	Expected effect	Evidence	Effort
1	Record hi/ta/Tanglish audio clips and write a verbatim true transcript for each; make the scorer Indic-safe (CER, transliterated WER, medical-term error rate)	`clinical_eval/raw_asr_score.py`, `fact_score.py`	No accuracy gain itself; makes every later number valid	Strong: peer-reviewed benchmark and MUCS 2021	M, dominated by recording and labelling

#	Change from today	Files	Expected effect	Evidence	Effort
2	Pass an explicit `language` per request; hi/ta/code-mix never reach MedASR; a cloud failure raises instead of falling back to an English engine	`asr.py`, `run.py`, `server.py`, `presets.py`	Removes today's silent wrong-engine path	Design reasoning plus the README's own caveat	S to M
3	Add Indic cloud engines and bake them off per language against the current chain	new adapter in `asr.py`	Largest lever for hi/ta; see the Tamil table below	Medium: one benchmark, phone speech not clinical	M to L
4	Request code-mixed output with English words in Latin script, so the existing English extractor and drug matcher keep working	adapter, `extract.normalize_text()`	Avoids building Hindi and Tamil lexicons for extraction	Low to medium: vendor docs, untested on drug names	S after change 3
5	Per-request keyterm list of 20 to 50 drug names drawn from the 71,872-brand reference	`drug_reference.py`, adapter	Targets drug-name misses, which the README calls the weak axis	Mechanism confirmed in vendor docs; no published gain for Indian speech	M
6	Run a numbers experiment: provider-normalized digits against verbatim words plus our own inverse text normalization	`extract.normalize_text()`	Targets misheard vitals and doses, the largest error source	None verified; an experiment, not a pick	M
7	Limit the LLM stages to choosing among ASR hypotheses and drug-reference candidates; later, fine-tune a corrector on review-gate corrections	`term_validate.py`, `context_repair.py`	Prevents WER regressions; a fine-tuned corrector gave about 21% relative gain on Mandarin-English	High for the risk, medium for the gain	S to audit, L to fine-tune

Decisions and alternatives

Each decision below lists the options, what the evidence says about each, and a recommendation. The recommendation is where I would start, not where I would stop: every engine choice should be confirmed on our own clips with true transcripts.

1. Which speech engine for hi-IN and ta-IN

On the Voice of India benchmark (536 hours of real phone conversations, 15 languages), Indic-specialist engines led and several global APIs failed outright. Tamil word error rate, lower is better:

Engine	Tamil WER (%)	Note
Sarvam Audio	14.2	Lowest WER in 13 of 15 languages
Sarvam Saarika 2.5	14.9	
Gemini 3 Pro	15.7	An LLM-based transcriber; see decision 5
AI4Bharat IndicConformer	19.9	Open weights; self-hostable
ElevenLabs Scribe v2	20.4	
Deepgram Nova-3	67.8	Tamil model added January 2026; the paper blames failed language detection and hallucination

Two cautions. The benchmark is telephone conversation, not clinical dictation, so the ranking may not carry over to drug names. Its true transcripts were first drafted by Whisper and IndicConformer models, which could favour those systems.

- **Option A, one best engine for everything.** Simplest to run, but no single engine led on every language, and English clinical dictation is a different problem.
- **Option B, a per-language routing table (recommended).** Choose the engine per language from our own bake-off; English may stay on MedASR or move to Nova-3 Medical, whose 6.79% medical keyword error rate is vendor-reported on undisclosed data.
- **Option C, self-host only.** IndicWhisper (Whisper fine-tuned on about 10.7K hours) won 39 of 59 Vistaar benchmarks in 2023, so fine-tuning works, but that result predates today's cloud engines. Keep IndicConformer as the fallback for air-gapped sites and outages.
- **Avoid by default:** Deepgram Nova-3 for Tamil, and AssemblyAI Universal or gpt-4o-mini-transcribe for any Indic language (over 100% WER on some). If one is used, always send an explicit language code.

2. What script the transcript is written in

The extractor reads English tokens. Its only Indic knowledge is 13 words, so the script choice decides how much new extraction code is needed.

- **Option A, Latin-script code-mix (recommended).** Sarvam's `codemix` mode keeps English words in Latin script and Indic words in native script, so drug names reach the existing matcher. The vendor shows one generic Hindi example and no drug-name or Tamil result; our bake-off must measure brand names such as Dolo and Pan-D.
- **Option B, native script.** Most faithful, but needs Hindi and Tamil lexicons and number handling in the extractor.
- **Option C, translate to English.** Convenient, but it returns no source-language text, so a reviewer cannot catch a dose or drug name the translation smoothed over. Offer it only as a second view beside the original.

3. How to get drug names right

- **Option A, per-request keyterm lists (recommended).** Deepgram, AssemblyAI and Sarvam Saaras v4 all accept them. Build a 20 to 50 term shortlist from the clinic's formulary and the first-pass fuzzy candidates, because the full 71,872-brand list will not fit. Published term caps conflict, so check current docs when building.
- **Option B, a word list in Whisper's prompt.** Measured harm on unmodified Whisper: it raised error on unlisted words on all 11 test sets and raised overall WER on 6 of 11, including a medical set (8.4 to 8.5). Keep the current short descriptive prompt.
- **Option C, fine-tune Whisper for prompt biasing.** B-Whisper cut out-of-vocabulary WER from 37.1 to 16.0 and overall WER from 12.1 to 8.9, but on English Common Voice with Whisper Large. Heavy, and unproven for Indic speech.
- **Already in place:** fuzzy drug repair against the reference. Keep it as the last line of defence.

4. Numbers, vitals and doses

Misheard numbers are the repo's largest error source, and no claim about spoken-number accuracy survived verification. Run it as an experiment: provider-normalized digits against verbatim words plus our own inverse text normalization, scored separately for blood pressure "X by Y", mg against mcg (a 1000 times error), other vitals and doses. EkStep's `indic-punct` has hand-written number grammars for 11 Indic languages, but its repository is over two years stale, so evaluate it before adopting.

5. What the LLM may do after speech recognition

- **Option A, choose only (recommended now).** Let the LLM pick among ASR hypotheses and drug-reference candidates. Constraining correction to the hypothesis list gave 9.03 to 8.14 WER out of domain, against 9.16 unconstrained.
- **Option B, free rewrite.** Unsafe: zero-shot GPT-3.5 raised Whisper's WER from 3.89 to 5.84, and GPT-4 to 4.60. Bigger models were not reliably better.
- **Option C, fine-tuned corrector, later.** A LoRA-tuned 7B model over Whisper's N-best list cut mixed error rate from 11.0% to 8.6% using 19 hours of pairs, but on Mandarin-English, not medical or Indic data. For Hindi, fine-tuned mT5 and ByT5 beat larger LLMs. This needs review-gate corrections to train on.
- **Option D, ensemble more engines.** Three engines combined beat Whisper alone by only 0.3 absolute; not worth the latency.

6. Where audio is processed

Cloud is allowed for this release, but the L7 plan also keeps an air-gapped deployment, and sending patient audio to a vendor has data-residency consequences that are not yet researched (see risks).
Recommendation: cloud primary behind a per-site flag, with self-hosted IndicConformer as the non-English fallback.

7. Audio-side processing

Keep denoising off. The README records that no denoiser has ever beaten `none` on clean clinical audio, and the denoising pilot's WER column cannot settle it because its no-denoise baseline already scored 1.20. I found no verified evidence on microphones, voice-activity chunking or doctor-patient diarization, so those are open.

Already changed in this session

Two small fixes are made and tested but not committed, on branch `chore/sanitize-isolate`. Both are prerequisites for the proposals above, and neither changes the English results.

Fix	Original code	Now	Check
Scorer tokenizer, `raw_asr_score.py`	Kept only `[a-z0-9]`, so every Devanagari and Tamil word was silently dropped and any hi/ta WER would cover English words only	Keeps letters, combining marks and digits in any script; normalizes Unicode forms; ignores joiner characters	Output identical to the old tokenizer on all 204 English texts in the shipped true-transcript and expected-fact files; 2 new tests
Unknown model names, `asr.py`, `run.py`, `server.py`	A typo such as `small` for `small-int8` silently ran Whisper `tiny`	`transcribe()` raises; the CLI rejects it as a usage error; the API returns 400	12 new tests, including that no decoder is built

Full suite after both: 470 passed, 20 skipped. The skips are the denoiser-arm tests that skip when scipy is absent, as the README documents.

How strong the evidence is

The research fetched 21 sources and extracted 103 claims, but only the top 25 were put to three independent verifiers each; 23 held and 2 were refuted. Treat the rest as unverified.

Strength	What it covers	Why
High	Free LLM rewriting of transcripts is unsafe; constrain it to the hypothesis list. Evaluation needs spelling-tolerant, transliteration-normalized WER	Two independent peer-reviewed sources each, unanimous votes
Medium	Engine ranking by language; Deepgram coverage; keyterm biasing; Whisper prompt harm; fine-tuned corrector gains	Mostly one source each; non-clinical or non-Indic data; vendor docs prove a feature exists, not that it works
Low	Latin-script code-mix keeps drug names for the English extractor	Split 2 to 1 vote; an inference from one generic Hindi example
Refuted	Two "100 terms" keyterm caps (Deepgram and AssemblyAI)	0 to 3 votes each; check current vendor docs
Not covered	Google Chirp 3, Azure, AWS Transcribe Medical, full gpt-4o-transcribe, NVIDIA Canary and Parakeet; MedASR on Indic; LoRA fine-tuning on small clinical sets; synthetic clinical speech; spoken numbers; microphones and diarization; cost	No surviving verified evidence, or not searched

All vendor versions are a mid-to-late 2026 snapshot (Saaras v4 is current and v2.5 is being deprecated), so rankings will move.

Risks and open questions

The first risk is a blocker, not a detail: sending patient audio to a cloud vendor is not cleared.

- **Data residency and vendor terms.** DPDP Act obligations, retention and training-use terms for patient audio were not researched. Written clearance must come before any cloud engine ships.
- **Invented clinical facts.** LLM-based transcribers such as Gemini can insert a drug or dose that was never said. Today's chain produced 0 spurious drug rows on the 30-clip set; gate every candidate on a count of drugs and doses absent from the reference.
- **Latency.** The target is p95 under 30 seconds for 2 minutes of audio. Sarvam's REST call takes at most 30 seconds of audio per request, so long clips need chunking; add cloud round-trips and the second pass, and measure it per engine.
- **Cost.** Not modelled. Prices quoted in the L7 plan are unverified.
- **Vendor churn.** Models and versions change quickly; keep the engine behind one adapter interface and pin the version used in each evaluation run.
- **Labelling effort.** Verbatim true transcripts in three languages need recorded speech and a reviewer fluent in each; this is the schedule driver, not the code.

Open questions the research could not answer:

1. Does Sarvam's code-mix mode keep Indian brand names (Dolo, Pan-D) and doses in Latin script on clinical Hinglish and Tanglish?
2. Which approach is most accurate for spoken numbers, blood pressure "X by Y" and mg against mcg in Hindi and Tamil?
3. How do Sarvam, Gemini, Google Chirp 3, gpt-4o-transcribe, ElevenLabs Scribe and Nova-3 Medical compare on Indian-accented clinical English? No verified clinical en-IN benchmark was found.
4. Can a hypothesis-constrained or fine-tuned corrector reduce drug and dose errors without adding hallucinated facts, and how much review-gate data does it need?

Proposed order and gates

Steps 2 and 3 can run in parallel; every later step waits for step 2.

1. **Settle the data-residency policy.** Gate: written clearance naming which vendors may receive patient audio.
2. **Build the true-transcript set and the Indic-safe scorer.** Gate: verbatim true transcripts for en-IN, hi-IN, ta-IN, Hinglish and Tanglish, at least the 30-clip size already used for English, and a baseline of the current chain on each.
3. **Add language routing with loud failure.** Gate: a test proving hi/ta audio cannot reach MedASR and that a cloud failure raises.
4. **Run the per-language engine bake-off.** Gate: an engine replaces the baseline only when a paired bootstrap interval beats it, with no drugs or doses inserted that are absent from the reference, and p95 latency inside budget.
5. **Add keyterm biasing and run the numbers experiment.** Gate: measured gains in drug-name recall and in blood-pressure, dose and unit errors, each with an interval.
6. **Shadow mode, then canary.** Run the chosen routing on live traffic beside the current chain and compare against clinicians' corrections at the review gate, as the L7 plan already specifies.
7. **Later, train a corrector** once the review gate has produced enough corrected transcripts.

Sources

Pages the research agents fetched, as of 2026-10-14. Repo claims cite file paths in the sections above.

- Voice of India benchmark (Interspeech 2026)
- IndicWhisper and the Vistaar benchmark
- Deepgram: Nova-3 adds 12 languages
- Deepgram: Nova-3 Medical
- AssemblyAI: prompting and keyterms
- Sarvam: selecting an output mode
- Sarvam: Saaras model page
- MUCS 2021 transliterated WER (Wiesner et al.)
- B-Whisper prompt biasing
- LLM error correction for code-switched ASR
- Constrained LLM transcript correction
- Hindi post-ASR correction (EACL 2026)
- indic-punct number grammars (seen in search results only, not opened)